

PROJECT AND TEAM INFORMATION

Project Title

PrioriTask: Energy-Based Intelligent Task Scheduling System

Student / Team Information

Team Name: Team #	TechBot Team T315
Team member 1 (Team Lead)	<i>Bisht, Manisha – 2510012646</i> <i>manishabishtsht8@gmail.com</i> 
Team member 2	<i>Choudhary, Purnima – 2510012541</i> <i>purnimachoudhary1307@gmail.com</i> 
Team member 3	<i>Bhardwaj, Yug – 2512500006</i> <i>yugbh2006@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Students often have several tasks to complete within a limited amount of time, including assignments, projects, revision, personal activities and other commitments. The main difficulty is not only maintaining a list of tasks, but deciding which task should be performed at a particular moment.

Traditional task lists generally depend on manually assigned priority and do not sufficiently consider factors such as the user's current energy level, available time, estimated effort or changing deadlines. As a result, a task with high priority may not always be the most appropriate task to perform at a particular moment.

PrioriTask aims to address this problem through an energy-aware and personalized task scheduling system. The system will use a C++ based core application with Data Structures and Object-Oriented Programming to organize and select tasks efficiently. A separate Python-based AI module is planned to analyse historical user actions and identify patterns in the user's behaviour.

The combination of algorithmic scheduling and personalized AI recommendations can help users make better decisions about what to work on and when to work on it.

State of the Art / Current Solution

Existing solutions mainly include traditional to-do list applications, calendar applications, reminder systems and priority-based productivity tools. These applications allow users to create tasks, assign deadlines and sometimes assign priority levels.

However, these approaches generally depend on manually entered information and predefined scheduling rules. A fixed priority order does not necessarily reflect the user's current ability or energy to perform a task.

Some productivity systems provide recommendations, but personalized recommendations based on a user's historical task-completion behaviour may be limited or unavailable. There is therefore an opportunity to combine task scheduling based on explicit task attributes with a separate AI layer capable of learning useful behavioural patterns from past actions.

PrioriTask proposes this combination while keeping the core scheduling logic understandable and implementable using fundamental DSA and OOP concepts.

Project Goals and Milestones

General Goals

1. Develop a functional task management application using C++.
2. Implement efficient task organization and selection using Data Structures.
3. Consider task priority, deadline, estimated effort and user's current energy level.
4. Design the system using Object-Oriented Programming concepts.
5. Maintain historical task-related information for future analysis.
6. Develop a separate Python-based AI recommendation module.
7. Analyse past user actions to identify useful behavioural patterns.
8. Integrate the AI module with the C++ application.
9. Evaluate the effectiveness of the resulting recommendations.

Initial Milestones

- Problem analysis and requirement identification
- Task attributes and scheduling logic design
- Selection of appropriate Data Structures
- C++ and OOP implementation
- Historical user-data design
- Python AI module development
- C++–Python integration
- Testing and evaluation

Project Approach

The proposed system will be developed as a modular application in which the C++ program forms the core scheduling system and Python provides an independent AI recommendation module.

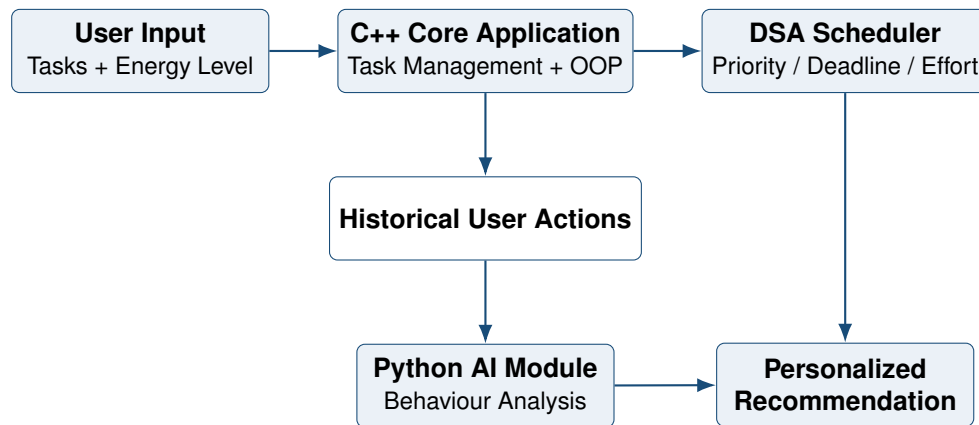
The C++ component will handle task creation, task storage, user input, energy-level information and scheduling operations. Data Structures such as arrays, linked lists and priority queues/heaps will be considered for efficient task management and selection. Object-Oriented Programming will be used to create modular components such as `Task`, `User` and `Scheduler`.

The scheduler will consider factors such as task priority, deadline, estimated effort, available time and current energy level. The result will provide an initial task recommendation.

The Python module will receive suitable historical task information from the C++ application. It will analyse previous user actions such as completed tasks, postponed tasks, task categories and completion patterns. Based on the available data, an appropriate machine-learning or recommendation approach will be explored.

The AI output will then be combined with the C++ scheduling result to provide a personalized recommendation. The AI component will act as an additional recommendation layer rather than replacing the core DSA-based scheduling algorithm.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project is expected to produce a functional task scheduling application capable of organizing tasks according to their priority, deadline, estimated effort and the user's current energy level.

The major deliverables will include:

- C++ based task management and scheduling application.
- DSA-based task organization and selection module.
- OOP-based classes and modular system architecture.
- Historical user-action storage mechanism.
- Python-based AI recommendation module.
- C++–Python integration mechanism.
- Personalized task suggestions based on available historical data.
- Testing results and project documentation.

The final system will demonstrate how fundamental DSA and OOP concepts can be combined with an AI component to develop a more adaptive and personalized productivity application.

Assumptions

The system assumes that users provide reasonably accurate task information, including priority, deadline and estimated effort. The current energy level is provided by the user when required. Historical behavioural recommendations will become more meaningful after sufficient task-related data has been collected. The initial AI module may therefore provide limited personalization until adequate historical data becomes available. The AI component is assumed to support, rather than replace, the deterministic DSA-based scheduling system.

References

1. C++ Programming Language and Standard Library documentation.

2. Data Structures and Algorithms course material.
3. Object-Oriented Programming with C++ course material.
4. Python documentation and standard library resources.
5. Scikit-learn documentation for potential machine-learning implementation.
6. Concepts related to priority queues, scheduling algorithms and recommendation systems.